Bachelor Thesis

Embedded AI Face Recognition with a Focus on Power Optimization Secure Deployment and System Performance

by

Muazzam Bin Aqeel

Submitted to the Department of Computer Science
in fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

at the

Technische Hochschule Ulm

Matriculation Number: 3139776

November 2025

Authored by:	Muazzam Bin Aqeel Department of Computer Science November 15, 2025
First Supervisor:	Manfred, Strahnen Professor of Computer Science
Secondary Supervisor:	Thorsten, Hasbargen Professor of Computer Science

THESIS COMMITTEE

Thesis Supervisors

Manfred Strahnen
Professor Doctor of Engineering
Department of Computer Science

Thorsten Hasbargen
Professor Doctor of Engineering
Department of Computer Science

Bachelor Thesis

Embedded AI Face Recognition with a Focus on Power Optimization Secure Deployment and System Performance

by

Muazzam Bin Aqeel

Submitted to the Department of Computer Science
on November 15, 2025 in fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

ABSTRACT

Face recognition has become a key enabler for modern access control systems, but most existing solutions rely on high-performance processors such as Raspberry Pi or GPU-accelerated devices. While effective, these platforms are not designed for continuous 24/7 operation under strict power constraints, making them unsuitable for low-power embedded applications such as battery-powered or energy-constrained door entry systems.

This thesis investigates the development of an AI-based face recognition system implemented on the STM32N6570 Discovery Kit, a microcontroller platform featuring an Arm Cortex-M55 core and a dedicated Neural-ART accelerator. The work focuses on adapting and optimizing a Python-based prototype into a fully embedded C/C++ application capable of running efficiently on the STM32 platform. The research covers multiple aspects: porting neural networks to the ST AI toolchain, designing a low-power runtime pipeline with sleep mode support, and integrating cryptographic functions to ensure secure storage and transmission of biometric data.

A comprehensive evaluation was conducted to compare the STM32N6570 implementation against the original Raspberry Pi-based prototype. Power consumption was measured across different operational states, including inference, idle, and deep sleep, using the board's built-in current measurement interfaces. Additional experiments examined the trade-off between inference accuracy and energy efficiency when deploying quantized neural networks.

The results demonstrate that low-power microcontrollers with AI accelerators can successfully perform real-time face recognition while significantly reducing energy consumption compared to Raspberry Pi systems. Furthermore, by integrating motion detection for conditional wake-up and hardware-accelerated cryptography, the system achieves both efficiency and robustness for long-term deployment.

This thesis shows that modern embedded AI platforms, such as the STM32N6 series, are viable candidates for practical biometric recognition applications where energy efficiency and security are critical. The findings contribute to the growing field of edge AI, where intelligent processing moves closer to sensors, enabling new classes of low-power smart devices.

Declaration

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Contents

Chapter 1

Introduction

This chapter introduces the motivation, objectives, and technical foundation of the work presented in this thesis. The focus is on developing an AI-based face recognition system on low-power microcontrollers, with particular attention to energy efficiency, cryptographic security, and system integration.

1.0.1 Motivation

Recent advancements in embedded AI have enabled real-time image recognition on resource-constrained hardware platforms. Unlike high-performance computing devices such as GPUs, microcontrollers with integrated AI accelerators can provide continuous operation with significantly reduced energy consumption. Such systems are critical in applications like access control, portable IoT devices, and always-on security systems.

$$\frac{\partial}{\partial t}\left[\rho\Big(e+|\vec{u}|^2/2\Big)\right]+\nabla\cdot\left[\rho\Big(h+|\vec{u}|^2/2\Big)\vec{u}\right]=-\nabla\cdot\vec{q}+\rho\vec{u}\cdot\vec{g}+\frac{\partial}{\partial x_j}\Big(d_{ji}u_i\Big) \tag{1.1}$$

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

1.0.2 Task

Chapter 2

Fundamentals

This chapter describes the hardware and software architecture used to realize an AI-based face recognition system on low-power microcontrollers. The focus is on modularity, scalability, and efficient integration of neural network inference with peripheral management.

2.0.1 STM32N6570-DK Board

2.0.2 Artificial Intelligence Face Dectection

2.0.3 Artificial Intelligence Face Recognition

2.0.4 FreeRTOS

Nomenclature for Chapter 2

C – Computational load M – Memory footprint E – Energy consumption

Chapter 3

Methodology

3.1 Requirements

The system is intended to be part of a door entry system that needs to be up and running 24 hours a day, 7 days a week. The goal is to achieve reliable face detection and recognition in real time while maintaining ultra-low power consumption suitable for continuous operation. The STM32N6570-DK board, featuring an Arm® Cortex®-M55 processor and Neural Processing Unit (NPU) accelerator, provides a powerful yet energy-efficient platform for deploying AI workloads at the edge. The system must be capable of capturing camera input, performing on-device inference for both face detection and face recognition, and controlling access mechanisms such as door locks or authentication signals all without relying on external computing resources.

System Requirements

The system requirements were derived from the user and developer stories defined during the project. These requirements summarize the functional and non-functional aspects of the face recognition system and provide a clear foundation for implementation and validation.

Functional Requirements

Graphical User Interface with Two-Step Authentication
A user interface provides the primary interaction between the user and the embedded system. It must be intuitive, visually responsive, and clearly structured despite limited hardware resources. The system integrates a graphical home interface that allows seamless navigation between the main face recognition mode and the administrative mode. After successful recognition, a secure PIN interface is displayed to enable a two-step authentication process, enhancing both usability and security for daily access operations.

Fast and Efficient Recognition
For a smooth user experience, real-time inference. The system performs both face detection and recognition locally on the STM32N6570-DK using the Neural Processing Unit (NPU) for acceleration. This ensures that recognition is achieved in less than two seconds per

user, providing quick and reliable access without dependency on external servers or network connections.

PIN and Multi-User Management
To support multiple authorized users, the system incorporates a secure PIN management feature within the administrative interface. Users can change their personal PINs directly on the device, ensuring confidentiality and flexibility. Furthermore, the system supports enrollment of multiple faces through an external configuration process, allowing easy management of user credentials in shared access scenarios.

Power-Saving Sleep Mode
Because the system operates continuously, minimizing idle power consumption is critical for long-term reliability. A dedicated sleep mode automatically activates after a defined period of inactivity, reducing system power draw. The board wakes instantly upon touch, ensuring responsiveness while maintaining ultra-low power operation suitable for embedded edge devices.

External Flash Management and Automation
External NOR flash memory is utilized for storing high-capacity assets such as user interface bitmaps, neural network weights, and firmware binaries. To simplify deployment and avoid manual flashing errors, a set of automated PowerShell scripts has to be developed. These tools allow efficient reading, analysis, and flashing of data to predefined memory regions, ensuring consistency and reproducibility across multiple hardware units.

Debug and Performance Monitoring
Comprehensive debugging and performance monitoring are vital for maintaining real-time reliability and optimizing system throughput. The implementation includes a UART-based logging interface that provides continuous runtime feedback for developers. Performance metrics, such as inference timing and system temperature, are monitored during execution to evaluate efficiency and hardware utilization.

On-Device Face Embedding Encryption
Since biometric data is highly sensitive, all stored face embeddings are encrypted using the AES-CBC algorithm to prevent unauthorized access or data extraction. Decryption and verification occur entirely on the device without cloud connectivity, ensuring that user privacy is fully preserved. This approach enhances security while maintaining fast local authentication performance.

Developer Requirements

External Flash Management and Automation
To manage large external assets such as images and AI model binaries, the system incorporates automated tools for reading, flashing, and verification of NOR flash memory. This process is executed through dedicated PowerShell scripts, reducing manual work and ensuring that each build deployment remains consistent across all development environments.

Debug and Performance Monitoring

During the development phase, visibility into real-time operations is crucial for troubleshooting and optimization. The system provides detailed UART-based debug output that logs critical events, such as frame rates, recognition timings, and power states. These logs help developers identify performance bottlenecks and ensure the NPU is effectively utilized for accelerated inference tasks.

Modular Code Architecture
A modular architecture allows independent development, testing, and maintenance of system components. The codebase is divided into clearly defined modules for the graphical interface, AI inference, cryptography, and system control. This separation reduces code dependencies, improves scalability, and simplifies the integration of new features or future hardware upgrades.

Table 3.1: Functional and Developer Requirements for the Face Recognition System

No.	Requirement
1	Graphical User Interface with Two-Step Authentication
2	Fast and Efficient Recognition
3	PIN and Multi-User Management
4	Power-Saving Sleep Mode
5	External Flash Management and Automation
6	Debug and Performance Monitoring
7	On-Device Face Embedding Encryption
8	Developer Flash Management Automation
9	Developer Debug and Performance Monitoring
10	Modular Code Architecture for Scalability

3.2 Concept and Design

3.2.1 Graphical User Interface

Graphical interfaces in embedded systems have become increasingly important as modern devices demand both functionality and user-friendly interaction. The growing availability of microcontrollers equipped with powerful display controllers and dedicated graphics accelerators has made it possible to implement visually rich and responsive interfaces even on low-power hardware.

The STM32N6570 DK development board represents this new generation of embedded platforms. The board's dedicated peripherals, including the LCD TFT Display Controller (LTDC), DMA2D (Chrom ART) graphics accelerator, and external NOR flash interface, enable the development of high performance display pipelines suitable for professional grade user interfaces.

In this thesis, different graphical interface development approaches were investigated to achieve an optimal balance between usability, responsiveness, and deterministic performance. The evaluated options included the official ST graphical framework TouchGFX, the commercial framework Embedded Wizard, and a fully custom C based GUI solution developed from the ground up. Each method provides specific advantages and trade-offs in terms of abstraction level, hardware control, debugging capability, and energy efficiency. The following sections explain these technologies and justify the final design choice.

TouchGFX

TouchGFX is a graphical framework developed and maintained by STMicroelectronics for STM32 microcontrollers. It integrates directly with STM32CubeIDE and provides a visual design environment that allows developers to create user interfaces using a drag and drop editor. The framework automatically generates C++ code for screen transitions, animations, widgets, and event handling. This greatly reduces development time and simplifies rapid prototyping, especially in projects that prioritize ease of use and visual design.

From a debugging and performance perspective, TouchGFX offers simulation tools and limited runtime performance monitoring through UART or ITM tracing. These tools allow observation of frame rates and rendering frequency. However, they operate only at the framework level and conceal the low-level mechanisms within the STM32N6570 DK display pipeline such as LTDC timing, DMA2D acceleration, and data transfers from external NOR flash memory. As a result, developers can observe overall rendering behavior but cannot analyze in detail how DMA2D transactions overlap with neural processing or how memory bandwidth influences refresh timing.

TouchGFX also manages its own rendering task at a high system priority, which can interfere with other FreeRTOS tasks. Because its internal scheduler is not user configurable, the framework may preempt time-critical operations such as AI inference, camera data acquisition, or communication services. Furthermore, integration with external trace tools like SEGGER SystemView or STM32CubeMonitor is limited because most timing-sensitive operations are handled internally. This lack of transparency and the complexity of integrating TouchGFX with the existing AI face recognition software made it unsuitable for this thesis project.

In summary, TouchGFX provides an excellent design experience and significantly accelerates GUI creation, but its internal abstraction prevents fine-grained performance analysis and precise control over hardware behavior. For an embedded AI system such as the STM32N6570 DK face recognition platform, where deterministic timing and synchronized task execution are essential, these limitations ruled out TouchGFX as a viable option.

Embedded Wizard

Embedded Wizard is a commercial graphical framework developed by TARA Systems GmbH for resource constrained embedded platforms. It combines a visual design studio with a proprietary runtime engine that generates optimized C source code for the selected hardware. The framework includes dedicated platform packages for STM32 devices, including the STM32N6 family, and integrates hardware acceleration features such as DMA2D and LTDC for efficient rendering.

Embedded Wizard provides high runtime efficiency and produces compact, optimized code that runs smoothly even on limited hardware. Its code generation process resolves layouts, animations, and resources at build time, resulting in predictable and consistent rendering performance. However, this high degree of automation also limits developer visibility into the underlying rendering pipeline. The framework does not expose detailed runtime information such as frame duration, DMA2D transfer latency, or LTDC synchronization intervals. Most performance data can only be observed through software-level debug messages in the Embedded Wizard Studio environment.

During evaluation on the STM32N6570 DK board, Embedded Wizard delivered stable performance and visually appealing results. However, its closed architecture and lack of access to hardware-level timing counters made it difficult to synchronize GUI updates with other tasks such as AI inference or camera processing. Because the framework manages its own rendering loop and task structure, it cannot be fully integrated with FreeRTOS timing mechanisms. This makes it unsuitable for systems that require deterministic operation and real-time coordination between the graphical interface and peripheral subsystems.

Another limitation is the framework's dependency on its proprietary build and deployment workflow. Extending or instrumenting its generated code for additional debugging or profiling requires modifications outside of the supported toolchain. For the face recognition application in this thesis, these restrictions were considered too limiting, as the project demanded deep control over DMA2D, LTDC, and memory interactions. Therefore, despite its professional design tools and visual quality, Embedded Wizard was not chosen because it does not provide the level of hardware transparency and flexibility required for performance-critical AI applications on the STM32N6570 DK platform.

Fully Custom GUI Framework

To overcome these constraints, a fully custom graphical user interface framework was developed entirely in C, specifically designed for the STM32N6570 DK hardware and FreeRTOS environment. This framework provides complete transparency and control over all graphical operations, from pixel drawing to full-screen refreshes, ensuring deterministic timing and smooth integration with AI inference and power management tasks. Every component of the display pipeline, including LTDC configuration, DMA2D acceleration, NOR flash access, and touch input, is explicitly defined in code.

This approach allows unrestricted access to hardware registers, interrupts, and performance counters. Developers can insert instrumentation points to measure draw call durations, DMA2D transfer latency, and LTDC refresh intervals using the DWT cycle counter and hardware timers. These measurements can be streamed over UART or stored for offline analysis, providing detailed insight into the system's timing behavior. This capability enables real-time identification of performance bottlenecks such as bus contention between DMA2D and the NPU or delays during bitmap transfers from external memory.

The framework also integrates directly with FreeRTOS task instrumentation, allowing visualization of scheduling, stack usage, and context switch timing. This ensures that graphical performance can be analyzed alongside other system components, making it possible to synchronize the GUI with AI inference and sensor tasks at a cycle-accurate level.

Unlike third-party frameworks, the custom implementation performs all rendering operations deterministically using direct hardware calls such as `UTIL_LCD_DrawBitmap()` and `UTIL_LCD_FillRect()`. All graphical resources are stored in predefined NOR flash memory addresses, ensuring a fully predictable memory footprint without dynamic allocation. This design eliminates fragmentation and allows consistent performance even under heavy system load.

An additional advantage of this design is the inclusion of real-time diagnostic overlays rendered directly on the display. These overlays can show live system metrics such as frame rate, task load, and memory usage, updated in the vertical blanking interval to avoid visual artifacts. Such on-screen feedback enables developers to tune system parameters during runtime without halting execution.

The framework's structure is based on a Finite State Machine (FSM) pattern, which organizes the user interface into distinct states such as Start, Admin, PIN Entry, Change PIN, and Face Recognition. Each state has its own display logic and transition rules, ensuring clear separation of functionality and predictable user navigation. This model provides deterministic control flow and simplifies debugging, as only one state is active at a time. The FSM structure is widely used in embedded software design for managing sequential and event-driven behavior, ensuring reliable and maintainable control of complex system interactions [1–3].

In conclusion, the custom GUI framework offers complete hardware awareness, full debugging transparency, and deterministic performance unmatched by existing graphical frameworks. It provides precise control over the STM32N6570 DK's graphics subsystem while maintaining synchronization with AI and peripheral tasks. This approach ensures the real-time reliability and efficiency required for the embedded face recognition system developed in this thesis.

3.2.2 Face Detection and Recognition

3.2.3 Power Consumption

3.2.4 Cryptography

3.3 System and Software Design

3.3.1 System Design

3.3.2 Software Design

Peripheral clocks were dynamically scaled. Face detection triggered wake-ups from STOP2 mode, ensuring responsiveness with minimal idle power.

Nomenclature for Chapter 3

T – Inf

Chapter 4

Results and Discussion

This chapter reports experimental results, analyzing performance, power consumption, and accuracy trade-offs.

4.1 Tests

4.1.1 Tests

Nomenclature for Chapter 4

A – Accuracy d – Embedding dimension q – Quantization level

Chapter 5

Evaluation

5.1 Discussion

5.1.1 Training Parameters

5.2 Outlook

Nomenclature for Chapter 5

FPS – Frames per second mW – Milliwatt

Appendix A

Code listing

This example uses the `listings` package.

```
function print_rate(kappa,xMin,xMax,npoints,option)
     local c = 1-kappa*kappa
     local croot = (1-kappa*kappa)^(1/2)
     local logx = math.log(xMin)
     local psi = 0

     local xstep = (math.log(xMax)-math.log(xMin))/(npoints-1)

     arg0 = math.sqrt(xMin/c)
     psi0 = (1/c)*math.exp((kappa*arg0)^2)*(erfc(kappa*arg0)-erfc(arg0))

     if option~=[[]] then
       tex.sprint("\\addplot+["..option.."] coordinates{")
       -- addplot+ for color cycle to work
     else
       tex.sprint("\\addplot+ coordinates{")
     end
     tex.sprint("("..xMin..","..psi0..")")

     for i=1, (npoints-1) do
       x = math.exp(logx + xstep)
       arg = math.sqrt(x/c)
       karg = kappa*arg
       if karg<5 then
     -- this break compensates for exp(karg^2), which multiplies the error in the erf approximation...
          logpsi = -math.log(croot) + karg^2 + math.log(erfc(karg)-erfc(arg))
          psi = math.exp(logpsi)
       else
```

```
          psi = (1/(karg) - 1/(2*(karg^3)) + 3/(4*(arg^5)) )/(1.77245385*croot)
          -- this is the large x asymptote of the reaction rate
       end
       logx = math.log(x)
       tex.sprint("("..x..","..psi..")")
     end
     tex.sprint("}")
end
\end{luacode*}
```

Appendix B

One-term coefficients for heat conduction

B.1 A multipage table of numbers

This example uses the `longtable` package: $\theta = A_1 f_1 \exp(-\lambda_1^2 \mathrm{Fo})$, $\overline{\theta} = D_1 \exp(-\lambda_1^2 \mathrm{Fo})$.

Table B.1: One-term coefficients for one-dimensional heat conduction with a convective boundary condition. Data follow H. D. Baehr and K. Stephan .

Bi	*Plate*			*Cylinder*			*Sphere*		
	λ_1	A_1	D_1	λ_1	A_1	D_1	λ_1	A_1	D_1
0.01	0.09983	1.0017	1.0000	0.14124	1.0025	1.0000	0.17303	1.0030	1.0000
0.02	0.14095	1.0033	1.0000	0.19950	1.0050	1.0000	0.24446	1.0060	1.0000
0.03	0.17234	1.0049	1.0000	0.24403	1.0075	1.0000	0.29910	1.0090	1.0000
0.04	0.19868	1.0066	1.0000	0.28143	1.0099	1.0000	0.34503	1.0120	1.0000
0.05	0.22176	1.0082	0.9999	0.31426	1.0124	0.9999	0.38537	1.0150	1.0000
0.06	0.24253	1.0098	0.9999	0.34383	1.0148	0.9999	0.42173	1.0179	0.9999
0.07	0.26153	1.0114	0.9999	0.37092	1.0173	0.9999	0.45506	1.0209	0.9999
0.08	0.27913	1.0130	0.9999	0.39603	1.0197	0.9999	0.48600	1.0239	0.9999
0.09	0.29557	1.0145	0.9998	0.41954	1.0222	0.9998	0.51497	1.0268	0.9999
0.10	0.31105	1.0161	0.9998	0.44168	1.0246	0.9998	0.54228	1.0298	0.9998
0.15	0.37788	1.0237	0.9995	0.53761	1.0365	0.9995	0.66086	1.0445	0.9996
0.20	0.43284	1.0311	0.9992	0.61697	1.0483	0.9992	0.75931	1.0592	0.9993
0.25	0.48009	1.0382	0.9988	0.68559	1.0598	0.9988	0.84473	1.0737	0.9990
0.30	0.52179	1.0450	0.9983	0.74646	1.0712	0.9983	0.92079	1.0880	0.9985
0.40	0.59324	1.0580	0.9971	0.85158	1.0931	0.9970	1.05279	1.1164	0.9974
0.50	0.65327	1.0701	0.9956	0.94077	1.1143	0.9954	1.16556	1.1441	0.9960
0.60	0.70507	1.0814	0.9940	1.01844	1.1345	0.9936	1.26440	1.1713	0.9944
0.70	0.75056	1.0918	0.9922	1.08725	1.1539	0.9916	1.35252	1.1978	0.9925
0.80	0.79103	1.1016	0.9903	1.14897	1.1724	0.9893	1.43203	1.2236	0.9904
0.90	0.82740	1.1107	0.9882	1.20484	1.1902	0.9869	1.50442	1.2488	0.9880
1.00	0.86033	1.1191	0.9861	1.25578	1.2071	0.9843	1.57080	1.2732	0.9855
1.10	0.89035	1.1270	0.9839	1.30251	1.2232	0.9815	1.63199	1.2970	0.9828
1.20	0.91785	1.1344	0.9817	1.34558	1.2387	0.9787	1.68868	1.3201	0.9800

Table B.1: (continued)

Bi	*Plate*			*Cylinder*			*Sphere*		
	λ_1	A_1	D_1	λ_1	A_1	D_1	λ_1	A_1	D_1
1.30	0.94316	1.1412	0.9794	1.38543	1.2533	0.9757	1.74140	1.3424	0.9770
1.40	0.96655	1.1477	0.9771	1.42246	1.2673	0.9727	1.79058	1.3640	0.9739
1.50	0.98824	1.1537	0.9748	1.45695	1.2807	0.9696	1.83660	1.3850	0.9707
1.60	1.00842	1.1593	0.9726	1.48917	1.2934	0.9665	1.87976	1.4052	0.9674
1.70	1.02725	1.1645	0.9703	1.51936	1.3055	0.9633	1.92035	1.4247	0.9640
1.80	1.04486	1.1695	0.9680	1.54769	1.3170	0.9601	1.95857	1.4436	0.9605
1.90	1.06136	1.1741	0.9658	1.57434	1.3279	0.9569	1.99465	1.4618	0.9570
2.00	1.07687	1.1785	0.9635	1.59945	1.3384	0.9537	2.02876	1.4793	0.9534
2.20	1.10524	1.1864	0.9592	1.64557	1.3578	0.9472	2.09166	1.5125	0.9462
2.40	1.13056	1.1934	0.9549	1.68691	1.3754	0.9408	2.14834	1.5433	0.9389
2.60	1.15330	1.1997	0.9509	1.72418	1.3914	0.9345	2.19967	1.5718	0.9316
2.80	1.17383	1.2052	0.9469	1.75794	1.4059	0.9284	2.24633	1.5982	0.9243
3.00	1.19246	1.2102	0.9431	1.78866	1.4191	0.9224	2.28893	1.6227	0.9171
3.50	1.23227	1.2206	0.9343	1.85449	1.4473	0.9081	2.38064	1.6761	0.8995
4.00	1.26459	1.2287	0.9264	1.90808	1.4698	0.8950	2.45564	1.7202	0.8830
4.50	1.29134	1.2351	0.9193	1.95248	1.4880	0.8830	2.51795	1.7567	0.8675
5.00	1.31384	1.2402	0.9130	1.98981	1.5029	0.8721	2.57043	1.7870	0.8533
6.00	1.34955	1.2479	0.9021	2.04901	1.5253	0.8532	2.65366	1.8338	0.8281
7.00	1.37662	1.2532	0.8932	2.09373	1.5411	0.8375	2.71646	1.8673	0.8069
8.00	1.39782	1.2570	0.8858	2.12864	1.5526	0.8244	2.76536	1.8920	0.7889
9.00	1.41487	1.2598	0.8796	2.15661	1.5611	0.8133	2.80443	1.9106	0.7737
10.00	1.42887	1.2620	0.8743	2.17950	1.5677	0.8039	2.83630	1.9249	0.7607
12.00	1.45050	1.2650	0.8658	2.21468	1.5769	0.7887	2.88509	1.9450	0.7397
14.00	1.46643	1.2669	0.8592	2.24044	1.5828	0.7770	2.92060	1.9581	0.7236
16.00	1.47864	1.2683	0.8541	2.26008	1.5869	0.7678	2.94756	1.9670	0.7109
18.00	1.48830	1.2692	0.8499	2.27556	1.5898	0.7603	2.96871	1.9734	0.7007
20.00	1.49613	1.2699	0.8464	2.28805	1.5919	0.7542	2.98572	1.9781	0.6922
25.00	1.51045	1.2710	0.8400	2.31080	1.5954	0.7427	3.01656	1.9856	0.6766
30.00	1.52017	1.2717	0.8355	2.32614	1.5973	0.7348	3.03724	1.9898	0.6658
35.00	1.52719	1.2721	0.8322	2.33719	1.5985	0.7290	3.05207	1.9924	0.6579
40.00	1.53250	1.2723	0.8296	2.34552	1.5993	0.7246	3.06321	1.9942	0.6519
50.00	1.54001	1.2727	0.8260	2.35724	1.6002	0.7183	3.07884	1.9962	0.6434
60.00	1.54505	1.2728	0.8235	2.36510	1.6007	0.7140	3.08928	1.9974	0.6376
80.00	1.55141	1.2730	0.8204	2.37496	1.6013	0.7085	3.10234	1.9985	0.6303
100.00	1.55525	1.2731	0.8185	2.38090	1.6015	0.7052	3.11019	1.9990	0.6259
200.00	1.56298	1.2732	0.8146	2.39283	1.6019	0.6985	3.12589	1.9998	0.6170
∞	1.57080	1.2732	0.8106	2.40483	1.6020	0.6917	3.14159	2.0000	0.6079

References

[1] S. Mazloum and K. Arabi. *Embedded Systems Design using the Finite State Machine Methodology*. Cham, Switzerland: Springer, 2019. DOI: 10.1007/978-3-030-15305-0.

[2] M. Balasubramanian and R. Arulmozhiyal. "Design and Implementation of Finite State Machines in Embedded System Applications." *International Journal of Advanced Research in Computer Engineering and Technology (IJARCET)*, **6**(2), 2017. Accessed October 17, 2025, pp. 169–174. ISSN: 2278-1323. URL: https://ijarcet.org/wp-content/uploads/IJARCET-VOL-6-ISSUE-2-169-174.pdf (visited on 10/17/2025).

[3] F. C. Morabito, V. Rovetto, and M. Versaci. "A Review on Finite State Machines and Statecharts in Embedded Software Design." *IEEE Access*, **6**(), 2018, pp. 29305–29318. DOI: 10.1109/ACCESS.2018.2839304.

[4] L. Bacchetta. "Implementation of a Software Interface Layer Between Model-Based Design and Embedded Graphic Frameworks (TouchGFX)." Accessed October 17, 2025. Master's thesis. Turin, Italy: Politecnico di Torino, 2021. URL: https://webthesis.biblio.polito.it/17898/1/tesi.pdf (visited on 10/17/2025).